Music Streaming Platform — Design Defense

Context

This document presents and defends the design decisions made for a simplified music streaming platform. The system allows users to browse music, manage personal playlists, and follow artists. Three use cases were modeled with sequence diagrams: **creating a playlist**, **adding a song to a playlist**, and **following an artist**.

---

1. Main Design Decisions

A single facade class: `StreamingPlatform`

Rather than modeling a separated frontend/backend architecture, we chose a single `StreamingPlatform` class acting as a **facade** (Facade pattern). It receives all requests from the web form via API calls, performs input validation, and delegates to the relevant domain classes.

This simplification was a deliberate assumption: the goal was to keep the model readable and focused on domain logic, not on infrastructure layers. In a real-world application, this class would be split into controllers, services, and repositories.

Static `create` and `get` methods on every data class

Every domain class (`Song`, `Album`, `Artist`, `User`, `Playlist`) exposes:

- a `+create(...)$ int` static method — encapsulates instantiation logic and returns the new record's technical id
- a `+get(id: int)$ T` static method — encapsulates retrieval by primary key

This is a simplified **Repository pattern**: each class is responsible for its own persistence operations. In a layered architecture, these methods would live in dedicated `SongRepository`, `PlaylistRepository`, etc. classes, but that level of separation was out of scope for this exercise.

Boolean return values

Unless otherwise stated, methods return a `bool` to indicate success or failure. This is a pragmatic choice for a simplified model. A more robust approach would use exceptions to force callers to handle error cases — a `bool` can silently be ignored.

Albums store `Song` objects, not foreign keys

All inter-class references use objects rather than raw ids. For example, `Album.songs` is typed `set[Song]`, not `set[int]`. This keeps the model in the **object-oriented domain** and avoids leaking relational database concerns into the class diagram. The conversion to foreign keys is a persistence layer concern, handled transparently by an ORM or repository.

---

2. How Responsibilities Are Distributed

Class	Responsibility
`StreamingPlatform`	Facade: input validation, orchestration, delegation
`User`	Manages own playlists and followed artists
`Playlist`	Manages its own song collection
`Album`	Manages its own song collection, with public batch methods and private unit methods
`Artist`	Represents a music creator; owns albums
`Song`	Atomic music unit; belongs to one album and one artist

A key principle is that `StreamingPlatform` **never manipulates data directly** — it validates, then delegates. For example, `createPlaylist` checks the name format (XSS prevention) before calling `Playlist::create`. This keeps domain logic inside domain classes.

---

3. Relationships and Multiplicities

Hierarchy: Artist → Album → Song

```
Artist "1" *-- "0..*" Album  : designs
Album  "1" *-- "0..*" Song   : contains
Song   "1" --> "1"    Artist : composed and interpreted by
```

- `Artist *-- Album` is a **composition**: an album cannot exist without its artist.
- `Album *-- Song` is a **composition**: a song belongs to exactly one album in this simplified model.
- `Song --> Artist` is a **directed association**: a song permanently references its artist. Note that this creates a shortcut alongside the `Album → Artist` path — this is intentional, as it allows direct access to the artist from a song without traversing the album.

> **Debate noted**: we initially discussed whether the hierarchy should be `Artist → Song → Album` (reflecting the creative process) or `Artist → Album → Song` (reflecting how streaming platforms structure published content). We chose the latter as it better represents the **published data model**, not the creative workflow.

> **Aggregation vs. composition for `Album *-- Song`**: in a real-world system, songs can exist independently (re-releases, compilations), which would make this an aggregation. We modeled it as composition as a simplification, and noted the trade-off explicitly.

User and Playlist

```
User "1"    *-- "0..*" Playlist : has exclusive
User "0..*" o-- "0..*" Artist   : follows
```

- `User *-- Playlist` is a **composition**: in this model, a playlist is private and owned by exactly one user. If shared playlists were introduced, this would become an aggregation with a many-to-many relationship.
- `User o-- Artist` is an **aggregation**: an artist exists independently of who follows them.

On the bidirectional design of `User ↔ Playlist`: `User` holds a `playlists: set[Playlist]` attribute, and `Playlist` holds an `owner: User` attribute. This bidirectionality is a deliberate choice for navigability — direct access from either side — rather than a data redundancy. In the relational database, only `Playlist.owner_id` would exist as a foreign key; the `User.playlists` attribute reflects association navigability, not a separate column.

Playlist and Song

```
Playlist "1" o-- "0..*" Song : references
```

- **Aggregation**: a song exists independently of any playlist. A playlist can also be empty (e.g. just created). Both sides justify the aggregation choice over composition.

StreamingPlatform

```
StreamingPlatform "1" --> "0..*" Album, Song, User, Playlist : uses
```

These are **directed associations**: `StreamingPlatform` references and manipulates all domain objects, but does not own their lifecycle. A composition would imply that no domain object could exist without the platform, which is semantically incorrect. No explicit relation to `Artist` was added, as no method on `StreamingPlatform` takes or returns an `Artist` object directly — artist-related filtering uses raw `str` parameters.

---

4. Alternatives Considered

`UserPlaylist` join class

We considered introducing a `UserPlaylist` association class between `User` and `Playlist`. We discarded it because **the relationship carries no data of its own** (no `added_at`, no `role`, no `is_pinned`). A join class is only justified when the relationship itself has attributes. Without them, it would only materialize a SQL join table — an implementation detail, not a domain concept.

It would become relevant if collaborative playlists were introduced, as the relationship would then need to carry a `role` attribute (owner, collaborator, viewer).

`song_id` vs `Song` objects as method parameters

We considered passing raw `song_id: int` values to methods like `Playlist::addSong` and `Album::addSong`. We chose to pass full `Song` objects instead, to keep the model at the **domain level** and enable validation directly within the receiving method. Passing ids would push ORM concerns into domain logic.

`Artist → Song → Album` vs `Artist → Album → Song`

Discussed above (see section 3). The `Artist → Album → Song` chain was chosen as it reflects the **published state** of content on a streaming platform, where a song is always released as part of an album (including singles, which are technically single-track albums).

`Album *-- Song` (composition) vs `Album o-- Song` (aggregation)

Composition was chosen for simplicity. In reality, a song can outlive its album (re-releases, compilations, artist deleting an album). The comment in the class diagram explicitly acknowledges this trade-off.

Separation of `addSong` / `addSongs` on `Album`

We chose to expose both a public batch method `+addSongs(list[Song]): int` and a private unit method `-addSong(Song): bool`. The private method holds all validation logic (duplicate check, format control); the public batch method simply iterates and delegates to it. This avoids duplicating validation logic and keeps the public API flexible.

---

5. Trade-offs

Decision	Advantage	Limitation
Single `StreamingPlatform` facade	Simple, readable, focused on domain	Not representative of a real layered architecture
`create`/`get` on domain classes	Self-contained, easy to read	Mixes domain and persistence concerns
`bool` return values	Simple to model and read	Failures can be silently ignored; exceptions would be safer
Composition for `Album *-- Song`	Clear ownership, consistent with simplified model	Does not reflect real-world flexibility (re-releases, compilations)
Composition for `User *-- Playlist`	Simple, reflects private ownership	Would need to become aggregation if shared playlists were introduced
Object references over ids	Clean OO model	Requires ORM translation layer in practice
No `Admin` / `Client` subdivision	Keeps model focused on the exercise scope	`roles: list[str]` on `User` is a placeholder only

6. What We Would Improve

- **Separate persistence from domain**: extract `create`/`get` methods into dedicated `Repository` classes (`PlaylistRepository`, `SongRepository`, etc.) to respect the Single Responsibility Principle.
- **Replace `bool` returns with exceptions**: force error handling at the call site rather than relying on callers to check return values.
- **Model `Admin` separately from `User`**: introduce a role-based access control model, or at minimum a proper `Admin` subclass, rather than a raw `roles: list[str]`.
- **Introduce a `Protocol` / interface**: common static methods (`create`, `get`, `delete`) could be grouped under a shared `Persistable` protocol, reducing redundancy across classes.
- **Make `Album o-- Song` an aggregation**: more honest with real-world constraints, at the cost of slightly looser ownership semantics.